

notificaciones [Ver todas las notificaciones](#)

 42

[Q](#)  
[yblanco-](#)

- [Gestionar franjas](#)
- [Configuración](#)
- [Cerrar sesión](#)

[¿Tienes un problema?](#)

- [Perfil](#)
- [Proyectos](#)
- [Aprendizaje en línea](#)
- [Foro](#)
- [Empresas](#)
- [Meta](#)
- [Tienda](#)

Menú Mis proyectos Gráfico Sagrado Listar proyectos  
[Disponibles Cursos](#) \_\_\_\_\_

## Escala para el proyecto [Llámame tal vez](#)

Debes evaluar a 1 estudiante de este equipo

Repositorio Git

[git@vogsphere-v2.42madrid.com](#)

## Introducción

- Manténgase educado, cortés, respetuoso y constructivo durante todo el proceso de evaluación. El bienestar de la comunidad depende de ello.
- Identifique con la persona (o el grupo) evaluado las posibles disfunciones del trabajo. Tómese el tiempo para discutir y debatir los problemas que ha identificado.
- Debe considerar que podría haber alguna diferencia en cómo sus compañeros podrían haber entendido las instrucciones del proyecto y el alcance de sus funcionalidades. Mantenga siempre una mente abierta y califique a él/ella con la mayor honestidad posible. La pedagogía es válida solo si la evaluación entre pares se realiza de manera seria.

## Directrices

- Califica únicamente el trabajo que está en el repositorio GiT del estudiante o del grupo.
  - Verifica de nuevo que el repositorio GiT pertenece al estudiante o al grupo. Asegúrate de que el trabajo sea para el proyecto relevante y también verifica que se utilice "git clone" en una carpeta vacía.
  - Verifica cuidadosamente que no se hayan utilizado alias maliciosos para engañarte y hacer que evalúes algo distinto al contenido del repositorio oficial.
  - Para evitar sorpresas, verifica cuidadosamente que tanto el estudiante evaluador como el evaluado hayan revisado los posibles scripts utilizados para facilitar la calificación.
  - Si el estudiante evaluador aún no ha completado ese proyecto en particular, es obligatorio que este estudiante lea todo el tema antes de iniciar la defensa.
  - Usa las señales disponibles en esta escala para indicar un repositorio vacío, programa que no funciona, un error de norma, fraude, etc. En estos casos, la calificación se termina y la nota final es 0 (o -42 en caso de fraude). Sin embargo, con la excepción de fraude, se recomienda continuar discutiendo tu trabajo (incluso si no lo has terminado) para identificar cualquier problema que pueda haber causado esta falla y evitar repetir el mismo error en el futuro.
  - Recuerda que durante la defensa, no debe haber segfaults ni otra terminación inesperada, prematura, incontrolada o inesperada del programa; de lo contrario, la nota final es 0. Utiliza la bandera adecuada. Nunca deberías tener que editar ningún archivo excepto el archivo de configuración si existe.
  - Si quieres editar un archivo, toma el tiempo para explicar las razones al estudiante evaluado y asegúrate de que ambos estén de acuerdo.
  - También debes verificar la ausencia de fugas de memoria. Cualquier memoria asignada en el montón debe liberarse correctamente antes del final de la ejecución. Se te permite usar cualquiera de las diferentes herramientas disponibles en la computadora, como valgrind, o e\_fence. En caso de fugas de memoria, marca la bandera correspondiente.
- 

## Adjuntos

[moulinette.zip](#)

[subject.pdf](#)

[data.zip](#)

[llm\\_sdk.zip](#)

## Parte obligatoria

### Preliminares

Comprueba los siguientes requisitos:

- Evalúa únicamente el trabajo que esté en el repositorio Git del estudiante o del grupo.
- El proyecto debe ejecutarse usando `uv run python -m src`
- Todos los errores deben gestionarse de forma adecuada, sin que se produzca un fallo
- Verifica que el JSON de salida siga el formato exacto especificado
- Comprueba que se implementa la decodificación con restricciones (no solo el prompting)
- Asegura una validez de JSON casi perfecta (salida 100% interpretable)

☐ Sí ☒ No

### Estructura del proyecto y dependencias

Verifica la configuración del proyecto:

- Ejecuta `uv sync` correctamente
- Verifica que `llm_sdk` esté correctamente integrado
- Comprueba que todas las clases usen `pydantic` para la validación
- Asegura que el programa se pueda ejecutar con `uv run python -m src`
- Verifica que la estructura del directorio `input/` sea correcta
- Comprueba que durante la ejecución se crea el directorio `output/`

☐ Sí ☒ No

### Gestión de archivos de entrada

Prueba el procesamiento de archivos de entrada:

- Verifica que el programa lea correctamente `function_calling_tests.json`
- Verifica que el programa lea correctamente `functions_definition.json`
- Prueba con JSON no válido en los archivos de entrada (debe gestionarse de forma adecuada)
- Prueba con archivos de entrada que falten (debe proporcionar mensajes de error claros)
- Verifica el manejo correcto de errores sin que se produzcan fallos

☐ Sí ☒ No

### Formato del archivo de salida

Verifica la corrección del archivo de salida:

- Comprueba que se crea el archivo de salida (predeterminado: `data/output/function_calling_results.json`, o la ruta proporcionada con `--output`)
- Verifica que el archivo contenga JSON 100% válido y recuperable (sin errores de sintaxis ni de análisis)
- Confirma que el JSON sigue estrictamente el esquema esperado
- Comprueba que cada entrada incluye exactamente: las claves `prompt`, `fn_name` y `args`
- Asegura que estén todos los argumentos requeridos y que coincidan con el esquema definido

- Verifique que los tipos de argumentos y los valores permitidos cumplan con las especificaciones de la función (p. ej., el campo de firmware restringido a opciones predefinidas)
- Confirme que no haya claves extra, texto o prosa fuera de la estructura JSON

☐ Sí ☒ No

### **Precisión de Llamada a Función**

Evalúe la precisión de las llamadas a funciones:

- Pruebe con indicaciones simples (p. ej., "agregar 2 y 3")
- Verifique la selección correcta de la función (se espera >90% de precisión)
- Verifique la precisión de la extracción de argumentos (se espera >90%)
- Pruebe con indicaciones ambiguas
- Verifique que el sistema maneje casos límite (cadenas vacías, números grandes)

¿La solución alcanza al menos 90% de precisión en la selección de funciones?

☐ Sí ☒ No

### **Uso del SDK de LLM**

Verifique el uso correcto del SDK de LLM:

- Verifique que encode y decode se usen correctamente
- Asegúrese de que no se accedan a métodos o atributos privados
- Confirme que se use el modelo Qwen/Qwen3-0.6B

☐ Sí ☒ No

### **Manejo de Errores y Robustez**

Pruebe el manejo de errores:

- Pruebe con JSON de entrada mal formado
- Pruebe con definiciones de funciones faltantes
- Pruebe con indicaciones que no coincidan con ninguna función
- Verifique que se proporcionen mensajes de error claros
- Asegúrese de que el programa nunca se bloquee inesperadamente

¿El programa maneja todos los casos de error con gracia?

☐ Sí ☒ No

### **Rendimiento y Confiabilidad**

Evalúe el rendimiento:

- Verifique que todas las indicaciones de prueba se procesen en un tiempo razonable (<5 minutos)
- Verifique que el 100% de las salidas sean JSON válidos (analizables)
- Verifique que la solución alcance >90% de precisión en las pruebas proporcionadas

- Verifique que el sistema sea confiable a través de múltiples ejecuciones

¿La solución cumple con estos criterios de rendimiento?

☐ Sí ☒ No

## Calidad de código y documentación

Revisión de la calidad del código:

- Verifique que el código esté bien organizado y legible
- Verifique el uso correcto de pydantic para validación
- Verifique que README.md explique claramente el algoritmo
- Verifique que README incluya decisiones de diseño y desafíos
- Verifique la presencia de indicaciones de tipo y documentación

¿La calidad del código y la documentación son satisfactorias?

☐ Sí ☒ No

## Evaluación Moulinette

Ejecute la evaluación Moulinette:

Siga estas instrucciones cuidadosamente:

1. Navegue al directorio moulinette
2. Ejecute uv sincrónico con éxito
3. Ejecute uv run python -m moulinette prepare\_exercises --set private con éxito
4. Ejecute uv run python -m moulinette grade\_student\_answers --set private --student\_answer\_path <ruta> con éxito
5. Verifique que la puntuación total sea mayor que 0
6. Verifique que la evaluación se complete sin errores

¿La evaluación Moulinette pasa con éxito?

☐ Sí ☒ No

## Bonificación

Verifique las características de bonificación (opcional, no requeridas para aprobar):

- Soporte para múltiples modelos LLM además de Qwen/Qwen3-0.6B
- Recodificación del tokenizador: no usar métodos de codificar y decodificar en el código principal, sino usar get\_logits\_from\_input\_ids y get\_path\_to\_vocabulary\_json.
- Mecanismos avanzados de recuperación de errores
- Optimizaciones de rendimiento (caching, batching)
- Suite de pruebas integral
- Visualización del proceso de generación
- Soporte para argumentos de funciones anidadas complejos
- Implementación pública de métodos de codificación de tokenizador y decodificación opcional

- Demostración de cómo la codificación y la decodificación se integran con la decodificación restringida

¿Se han implementado características adicionales notables?

Califúcelo de 0 (fallido) a 5 (excelente)

---

## Calificaciones

No olvide marcar la bandera correspondiente a la defensa

Ok Proyecto sobresaliente

Trabajo vacío Trabajo incompleto Compilación inválida Norma Trampa Colapso Situación preocupante Función prohibida No puede soportar / explicar el código

## Conclusión

Bandera

☒ Ok ☐ Trabajo incompleto ☐ Compilación inválida ☐ Norma Trampa ☐ Colapso ☐ Sobresaliente proyecto ☐ Situación preocupante ☐ Función prohibida ☐ No puede soportar / explicar el código ☐ Trabajo vacío

Deja un comentario sobre esta evaluación (máx. 2048 caracteres)

Terminar evaluación

[Términos generales de uso de la API](#)[Declaración sobre el uso de cookies](#)[Política de privacidad](#)[Término general de uso del sitio](#)[Reglamento](#)[Procedimiento](#)[Términos de uso para videovigilancia](#)[Avisos legales](#)

x

Cancelar Enviar

x

**Contenido modal de Flash (crudo)**

Cerrar